ECE8803-AI4
Module: Fault Detection and Classification
Lecture 2: Multivariate Statistical Process Control

Course website: https://aikhan123.github.io/khan-lab-website/ai-semiconductors-course.html

Asif Khan
Associate Professor
School of Electrical and Computer Engineering
School of Materials Science and Engineering
Georgia Institute of Technology
akhan40@gatech.edu

Georgia Tech

Where Lecture 1 ended

We have a model of normal and two repaired detectors. We have no way to say whether either one is any good.

What exists

A baseline. One hundred healthy runs, aligned, summarised by a mean run and three eigentraces.

A compact code. Three or four numbers reproduce a healthy run to 0.3% of its span.

Two detectors. Residual against the golden trace, and distance in the space of coefficients. Neither fires on a healthy run any more.

What is missing

A threshold with a reason. The ±20 in Lecture 1 was picked by hand.

A score for a new run. One number, computed the same way every time.

Evidence. No faulty run has been scored, so the detection rate is unknown.

100	3	0	±20
aligned healthy runs	eigentraces kept	faults scored so far	threshold, chosen by hand

Georgia Tech

The six steps of an FDC system

Lecture 1 covered the first two. Today covers the next three.

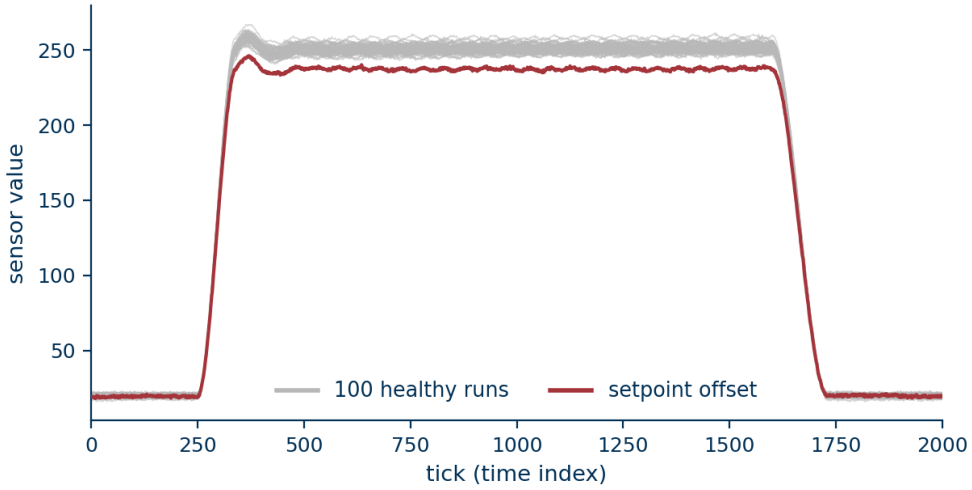
1	**Segment**	cut the trace to one recipe step, using context data	context
2	**Align**	remove the start-time variation	Lecture 1
3	**Model**	fit a baseline on known-good runs only	Lecture 1
4	**Detect**	score each new run against control limits	**today**
5	**Diagnose**	attribute the alarm to a part of the run	**today**
6	**Act**	route it to an engineer through an action plan	**today**

Segmentation and the action plan are engineering rather than modelling, and they are where most deployments actually fail.

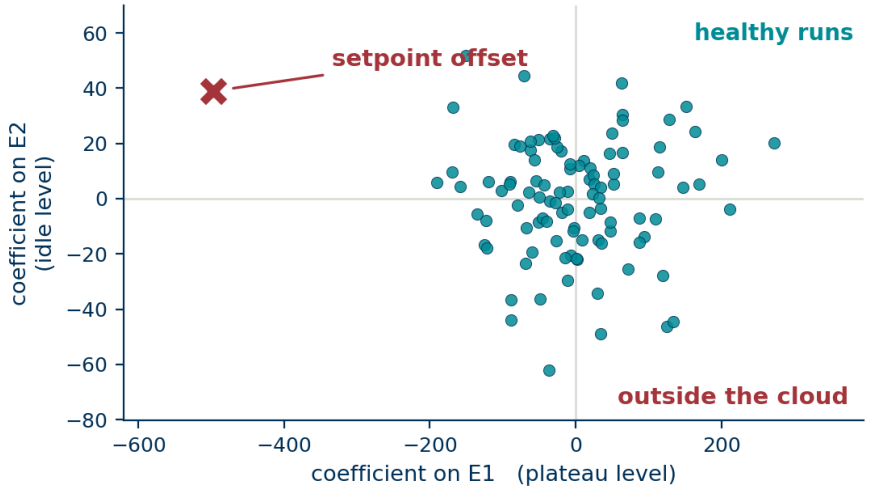
Give the model a run it has never seen

The setpoint-offset run from Lecture 1. Align it, subtract the same mean, project onto the same three eigentraces.

the new run, aligned and overlaid on the baseline



its coefficients, against the healthy population



Its coefficients

E1 = −497 against a healthy spread of 88. That is about five and a half standard deviations from the centre.

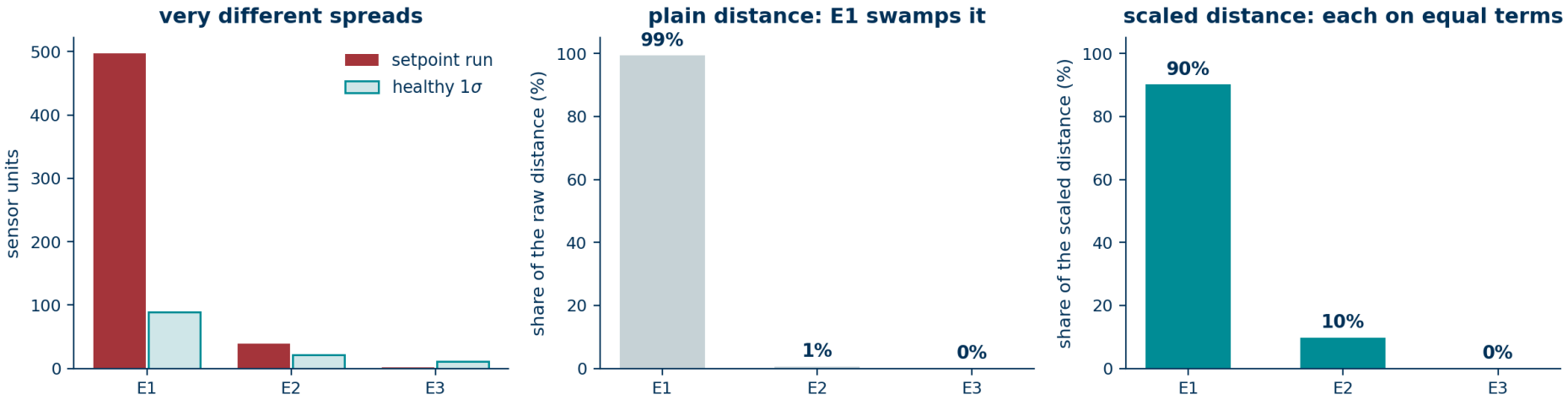
First use of a label

The model is still built from healthy runs only. The label on this run is being used to test, not to fit.

Georgia Tech

Turn that picture into one number

A detector needs a rule. The obvious one is distance from the centre of the healthy cloud.



Plain distance does not work

The components have very different spreads: 88, 21 and 11 units. Squaring and adding lets E1 account for 99% of the distance, so a large deviation on E3 is invisible.

Scale each one first

Divide every coefficient by the standard deviation that component has among healthy runs, then sum the squares. Every direction now counts on equal terms.

Georgia Tech

That number is Hotelling's T^2

The scaled squared distance from the centre of the healthy cloud, inside the space the model spans.

$$T^2 = \Sigma \; t_a^2 \; / \; \lambda_a \qquad \text{summed over the } a = 1 \ldots k \text{ components}$$

What the two symbols are

t_a the run's coefficient on component a. With k = 3 there are three of them, and they are all T^2 ever sees.

λ_a the variance of that component among the training runs, which is the scaling

the whole statistic

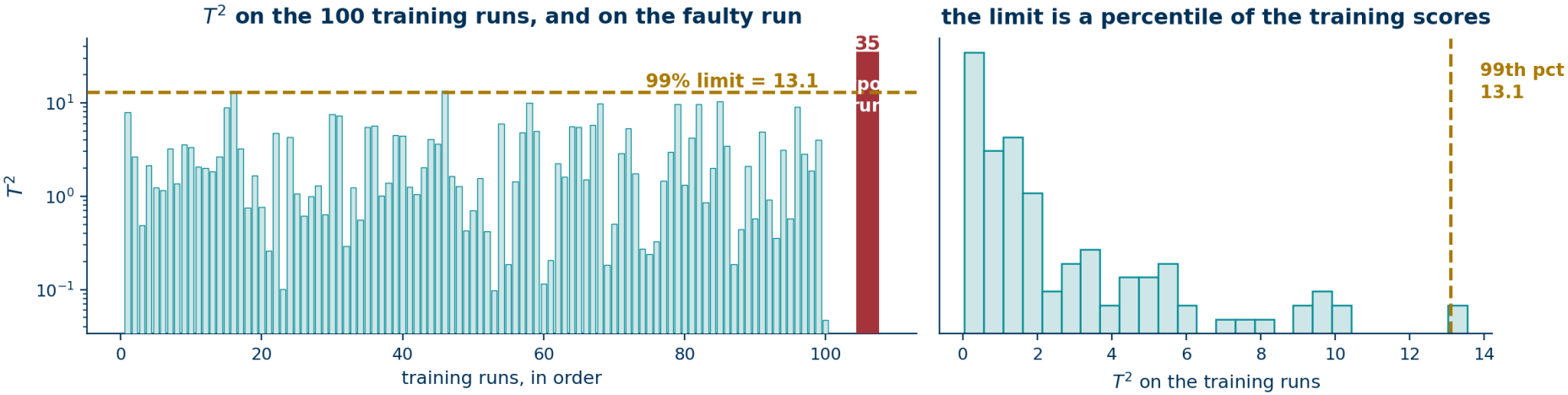
```
mu  = Xgood.mean(0)
U,s,Vt = np.linalg.svd(Xgood - mu)
P   = Vt[:k].T
lam = (s**2 / (n-1))[:k]
T   = (x - mu) @ P
t2  = ((T**2) / lam).sum()
```

A large T^2 says the run is built out of familiar patterns, in an amount the healthy population never produced. The setpoint run scores 35.

Georgia Tech

Setting the limit

Score the training runs, and put the limit at a percentile of those scores.



The rule

Take the 99th percentile of the training T² values. Here that is 13.1. By construction about one training run in a hundred sits above it.

This replaces the hand-picked ±20

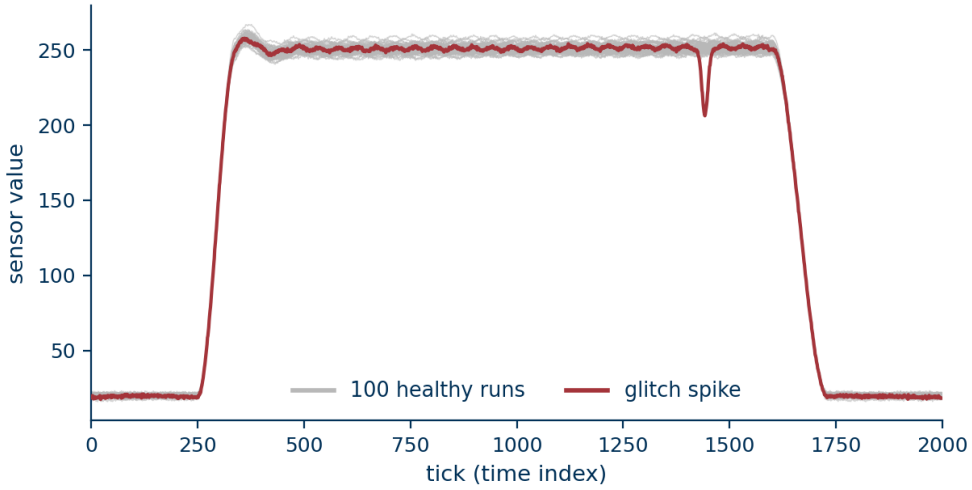
The limit now comes from the data and moves with it. Re-fit the baseline after maintenance and the limit re-derives itself.

Georgia Tech

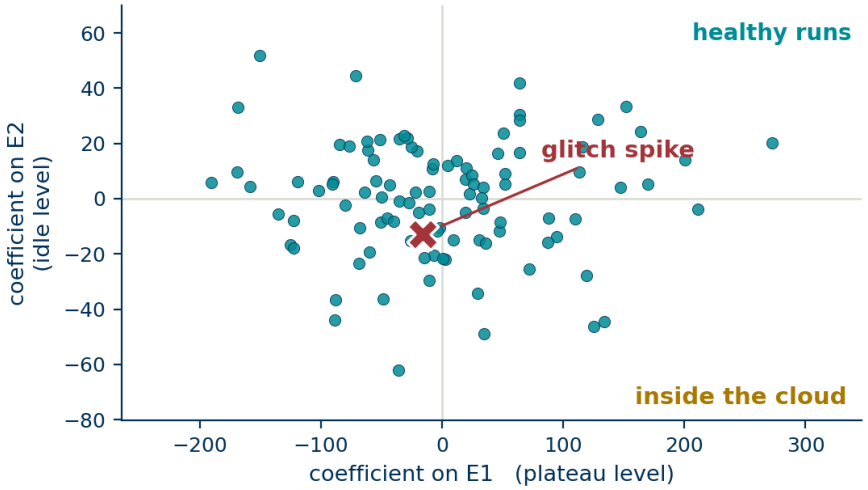
Now give it a different fault

A glitch spike: a brief disturbance on the plateau, about forty units deep and a dozen ticks wide.

the new run, aligned and overlaid on the baseline



its coefficients, against the healthy population



The coefficients look ordinary

E1 = −16, E2 = −13, E3 = −11, all well inside the healthy range. T² comes out at 2.7 against a limit of 13.1.

The run passes

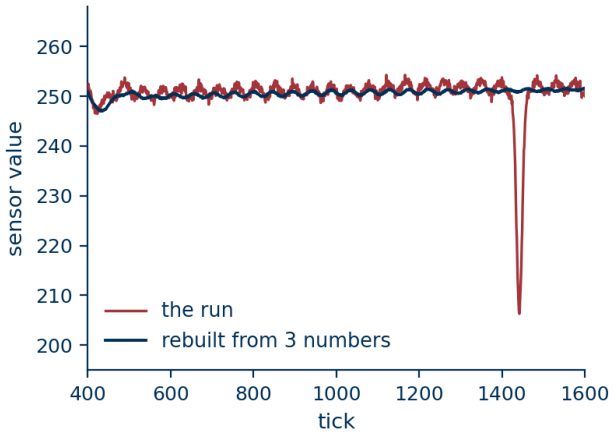
A fault that would scrap wafers sits in the middle of the healthy cloud.

Georgia Tech

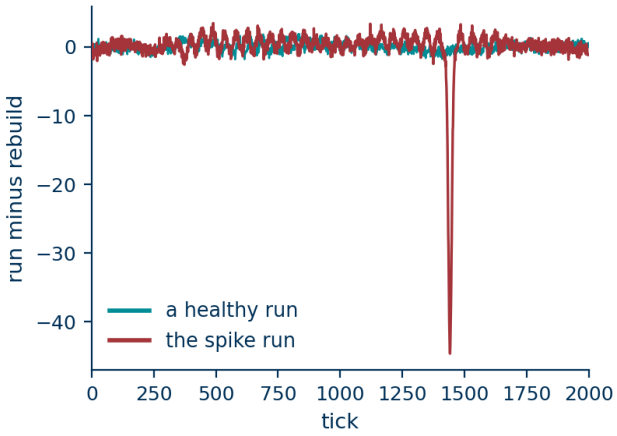
Why the spike got through

Rebuild the run from its three coefficients and compare it with the original.

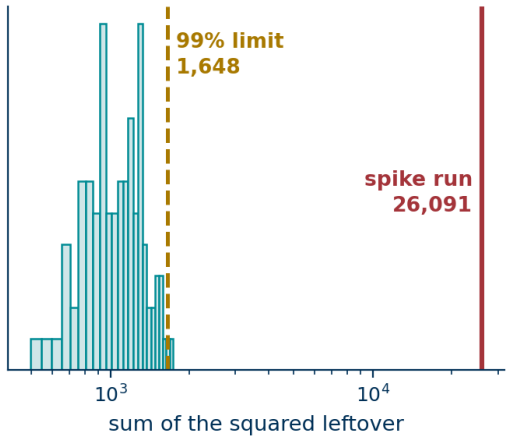
the spike does not survive the rebuild



what the model could not rebuild



that leftover, measured



The rebuild is smooth

Nothing in the three eigentraces looks like a twelve-tick notch, so the projection cannot represent one. The coefficients stay ordinary because the spike never reached them.

The evidence is what was left behind

Subtract the rebuild from the run. A healthy run leaves noise. The spike run leaves the spike.

Georgia Tech

That leftover is the second statistic

Square it, sum it over the run, and you have a number that measures how much of the run the model could not account for.

$$\text{SPE} = \|x - \hat{x}\|^2 \qquad \text{where} \quad \hat{x} = \text{mean} + P\,P^T\,(x - \text{mean})$$

x the run, 2000 numbers · P the three eigentraces, 2000 × 3 · $\hat{x}$ the rebuilt run, 2000 numbers

T^2 and SPE live in different spaces

T^2 is computed on the 3 coefficients.

SPE is computed on the 2000 samples.

That is why the two carry independent information, and why a run can be ordinary in one and extreme in the other.

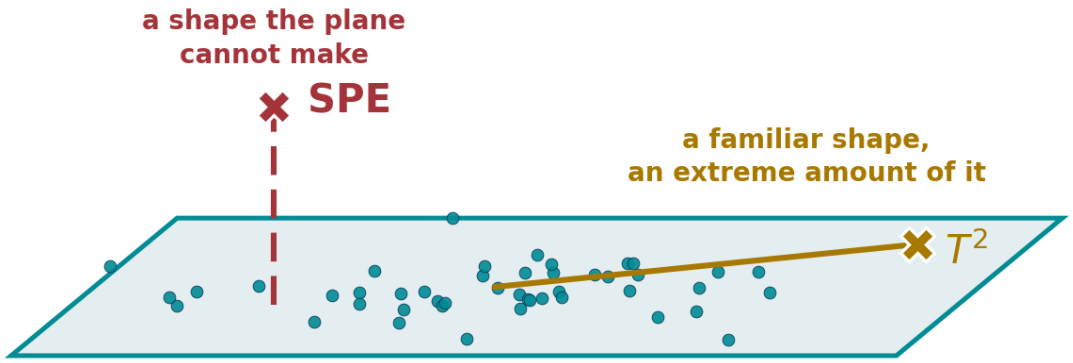
two more lines, with shapes

```
z    = x - mu                # 2000
t    = z @ P                 # 3
xhat = mu + t @ P.T          # 2000
spe  = ((x - xhat)**2).sum() # 1
```

Lecture 1 slide 24 measured this same quantity: a square pulse rebuilt 7% wrong and pure noise 23% wrong, against 0.3% for a real run. On the spike run SPE is 26,091 against a limit of 1,648.

Two ways to be abnormal

The healthy runs occupy a thin slab inside a 2000-dimensional space. A new run can leave it in two independent ways.



T^2 measures distance travelled INSIDE the plane, scaled by how far healthy runs normally travel in each direction. SPE measures distance OFF the plane, which is whatever the three eigentraces could not account for.

T^2 · inside the plane

The run is made of familiar ingredients in an unusual amount. Level shifts, gain errors and drift land here.

SPE · off the plane

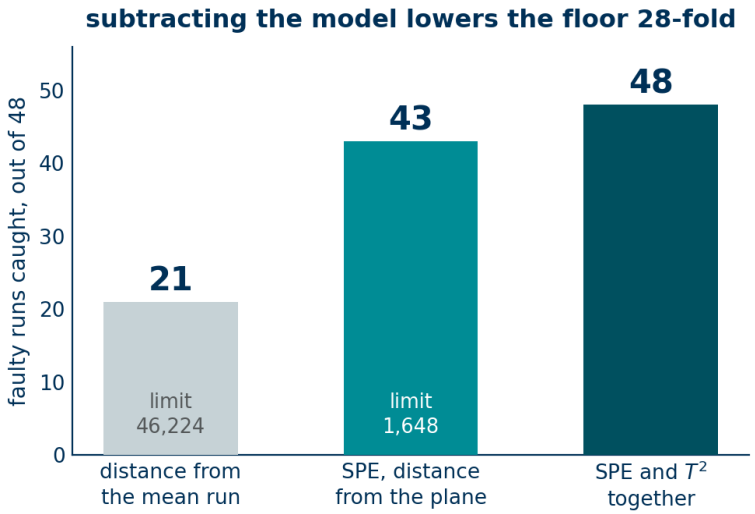
The run has a shape the model cannot produce at all. Spikes, noise bursts and novel failure modes land here.

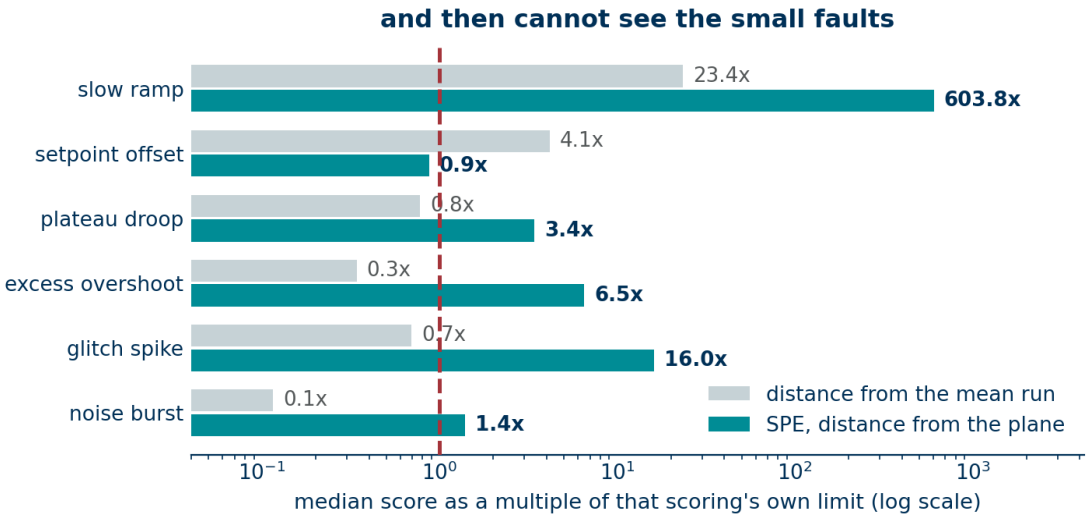
A fault that is invisible to one statistic is usually loud in the other, which is why production systems monitor both.

Georgia Tech

Why subtract the model first

SPE works on the 2000 samples, so the obvious question is why we needed the eigentraces. Score the raw distance from the mean run and compare.





What the healthy runs cost you

A healthy run differs from the mean run by its own gain, offset and drift. That legitimate variation sets the limit at 46,224 and buries everything smaller.

What the projection removes

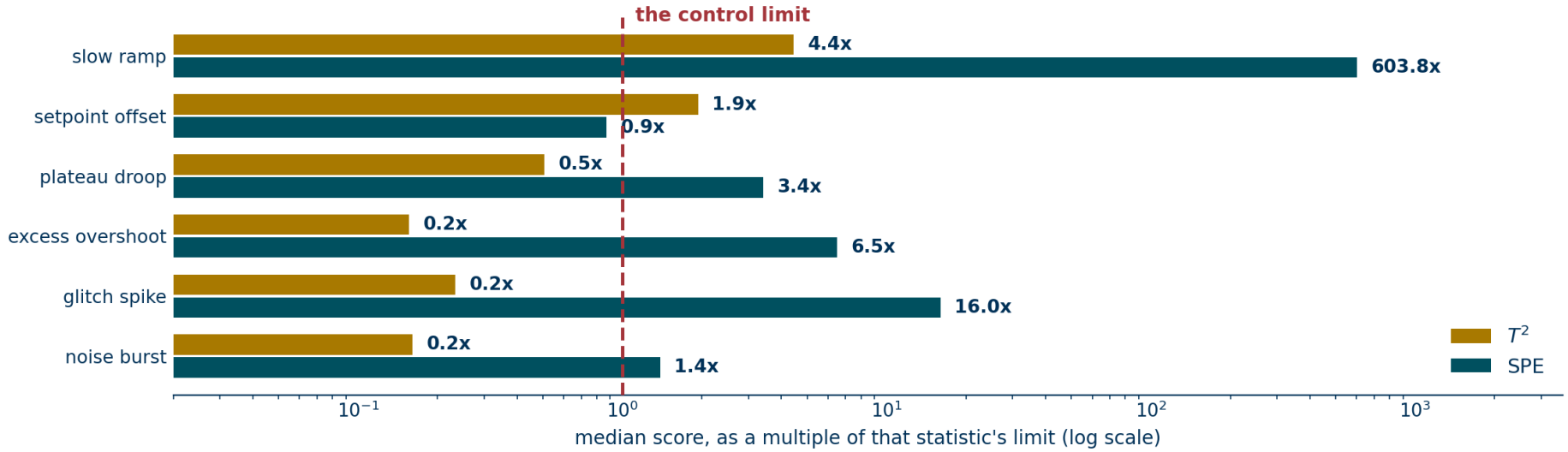
$\hat{x}$ is the best the model can do using healthy patterns alone, so $x - \hat{x}$ contains only what those patterns cannot explain. The limit falls to 1,648.

Setpoint offset moves the other way, from 4.1x to 0.9x, because the plateau level is a direction the model owns and the projection absorbs it. That is the gap T^2 fills.

Georgia Tech

Which statistic catches which fault

The six fault types from Lecture 1, scored with both statistics. Bars are the median score as a multiple of that statistic's own limit.



Level faults need T²

Setpoint offset scores 1.9x on T² and 0.9x on SPE. The offset is a shape the model owns, so it hides inside the plane.

Shape faults need SPE

The glitch spike scores 0.2x on T² and 16x on SPE. Slow ramp reaches 600x, because a changed edge is far outside the model.

Georgia Tech

The experiment that produces a number

Labels enter here, and only here. They are used to score the detector after it is built.

How the faulty runs are made

One defect per run. Injected into a run that had already passed the health gate, so a faulty run is never also an unusual healthy run.

Severity is a knob. Each fault type has a nominal magnitude, and it can be swept from zero upwards.

Six types, eight runs each. Forty-eight faulty runs in total.

How the runs are split

100 healthy runs the Lecture 1 baseline. These fit the mean, the eigentraces and both limits.

40 healthy runs same tool, same generator and seed, never fitted

48 faulty runs every fault type, at nominal severity

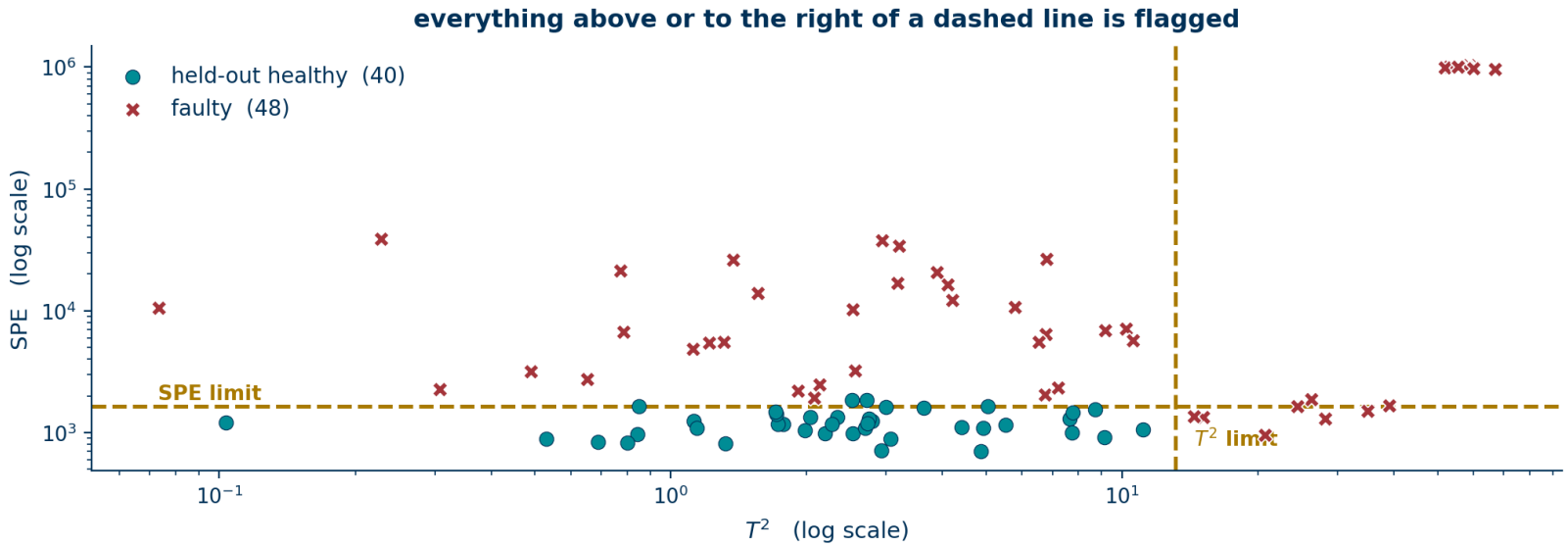
The model never sees a faulty run at any point.

Fitting on the runs you later score would report a detection rate that no deployment could reproduce.

Georgia Tech

Results on the held-out runs

Every run scored on both statistics, with the two limits drawn as dashed lines.



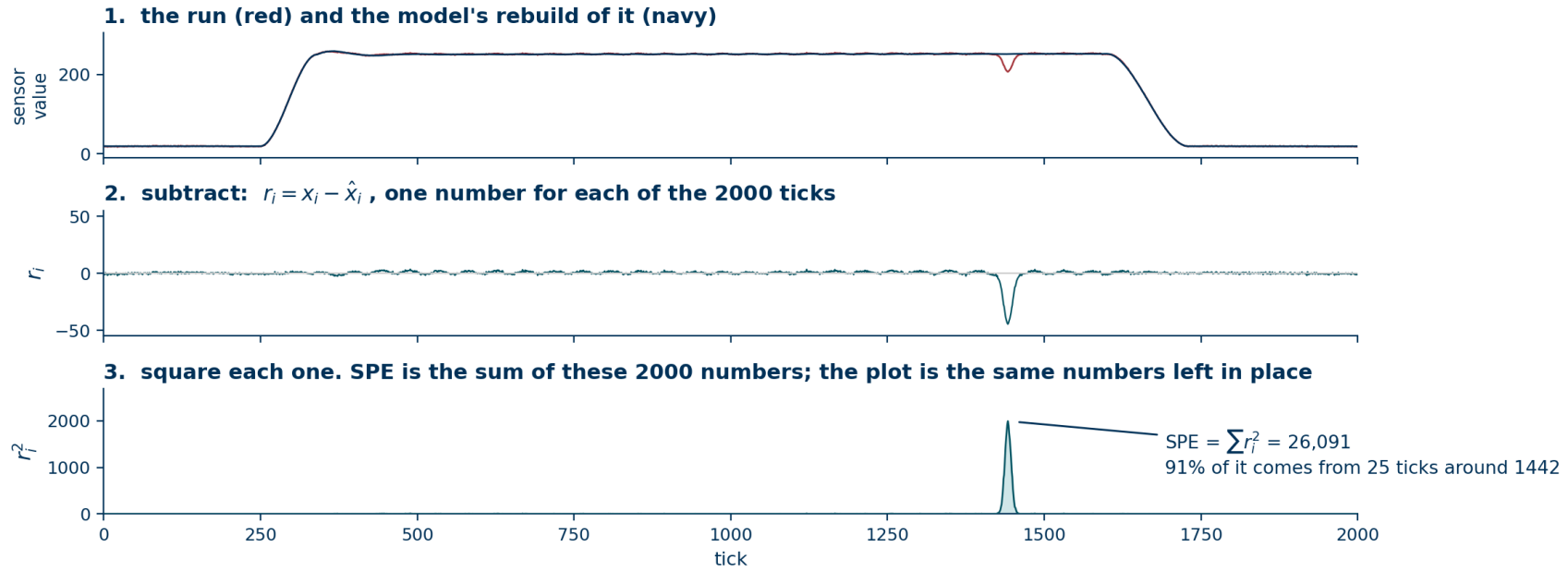
48 / 48	3 / 40	32	3
faulty runs caught	healthy runs flagged, against the 1% promised	faults that only SPE catches	eigentraces in the model

Every fault is caught. The false-alarm rate comes out at 7.5% rather than the 1% the percentile promised, and the slide on baseline size explains why.

Georgia Tech

How a contribution plot is made

SPE adds up 2000 squared residuals. The plot is those same 2000 numbers, kept in place instead of added up.



Nothing extra is computed

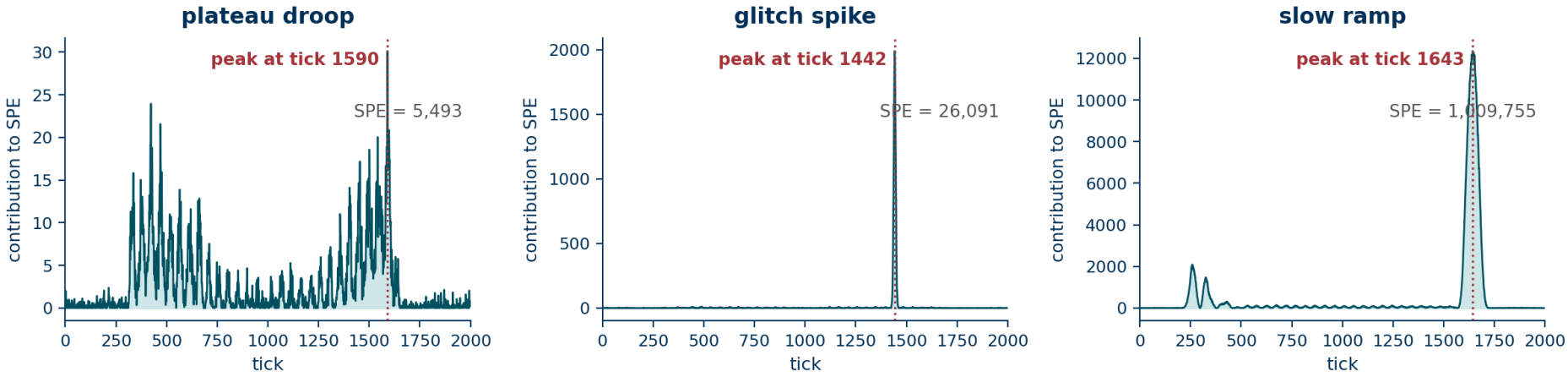
Every term was already there. Summing them produced SPE; not summing them produces the plot.

The peak names the tick

Ninety-one percent of this run's SPE comes from twenty-five ticks around tick 1442.

Where in the run did the alarm come from

SPE is a sum over ticks. Keep the individual terms instead of the total and you get a plot against time.



This is the C in FDC

An alarm that says run 4471 is abnormal closes no tickets. An alarm that says run 4471 is abnormal at the plateau points at the heater.

One caveat

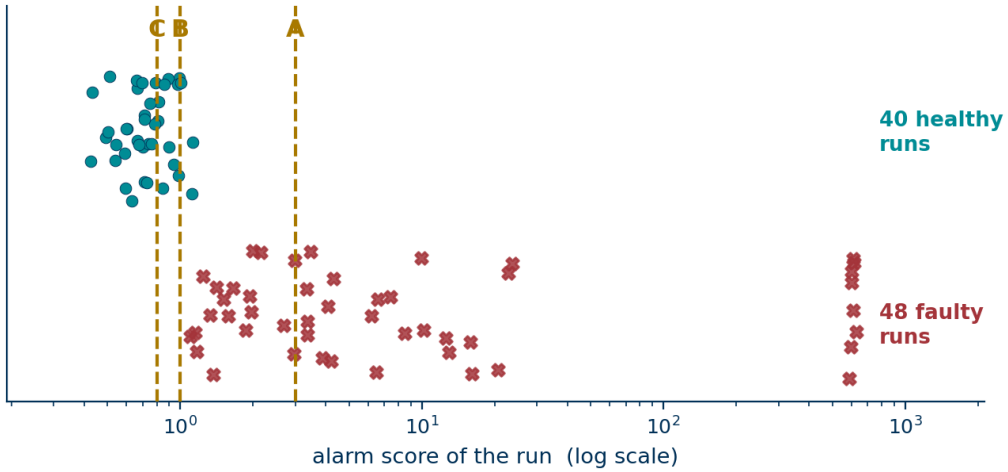
Contributions are located in the aligned time frame. To report a tick number to an engineer, undo the shift first.

Georgia Tech

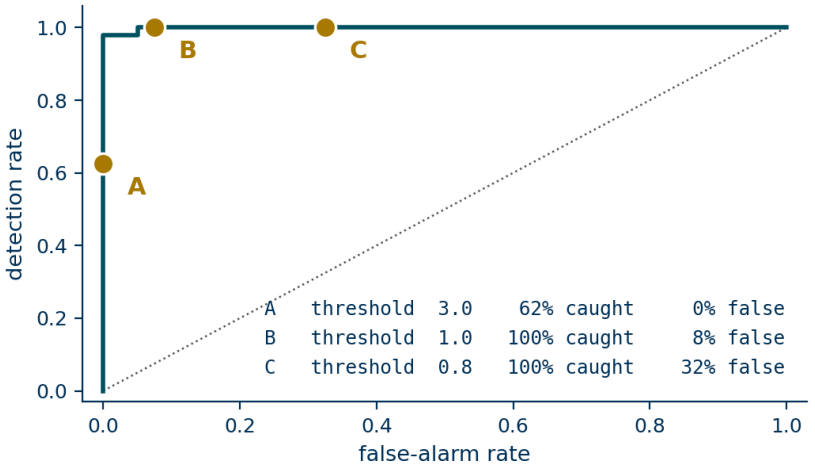
How the ROC curve is drawn

Each run already has one alarm score. Pick a threshold, count what falls above it, and you have one point.

every run has one score; a threshold splits the line



one threshold, one point. AUC = 0.999



One threshold, one point

Above the line at B: every faulty run, and three of the forty healthy ones. That pair, 100% and 8%, is the dot marked B.

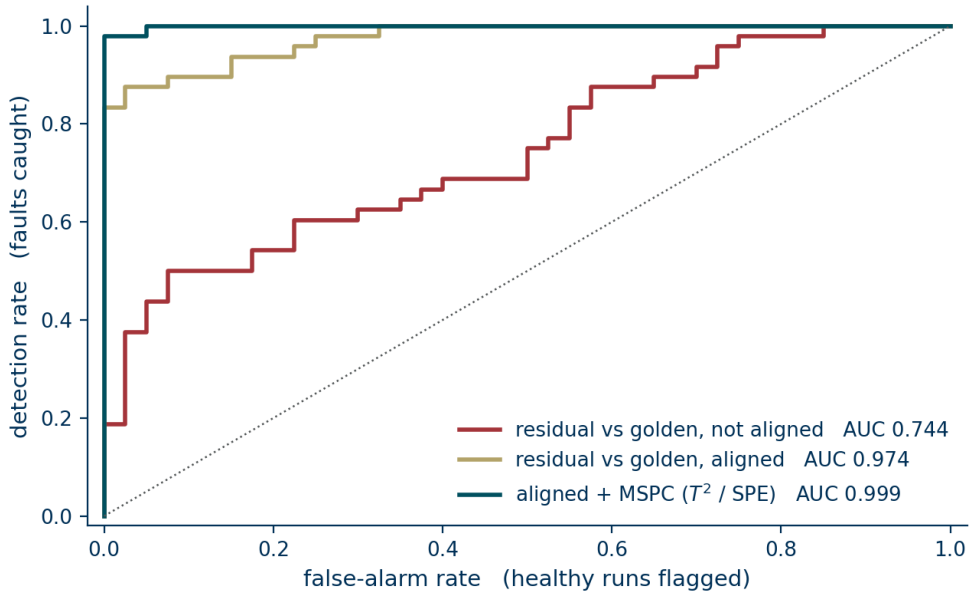
Sweep it and you get the curve

Strict at A, the deployed limit at B, loose at C. The area under the whole sweep is one number that needs no threshold.

Georgia Tech

Three detectors on the same held-out set

The naive residual from Lecture 1, the same residual after alignment, and the two MSPC statistics together.



Area under the ROC curve

0.744 residual against the golden trace, no alignment

0.974 the same rule, applied to aligned runs

0.999 aligned, with T² and SPE together

The step from the first to the second cost one integer per run. The step from the second to the third cost three eigentraces and two percentiles.

Georgia Tech

The four outcomes, and what each one costs

Any detector puts every run into one of these four boxes.

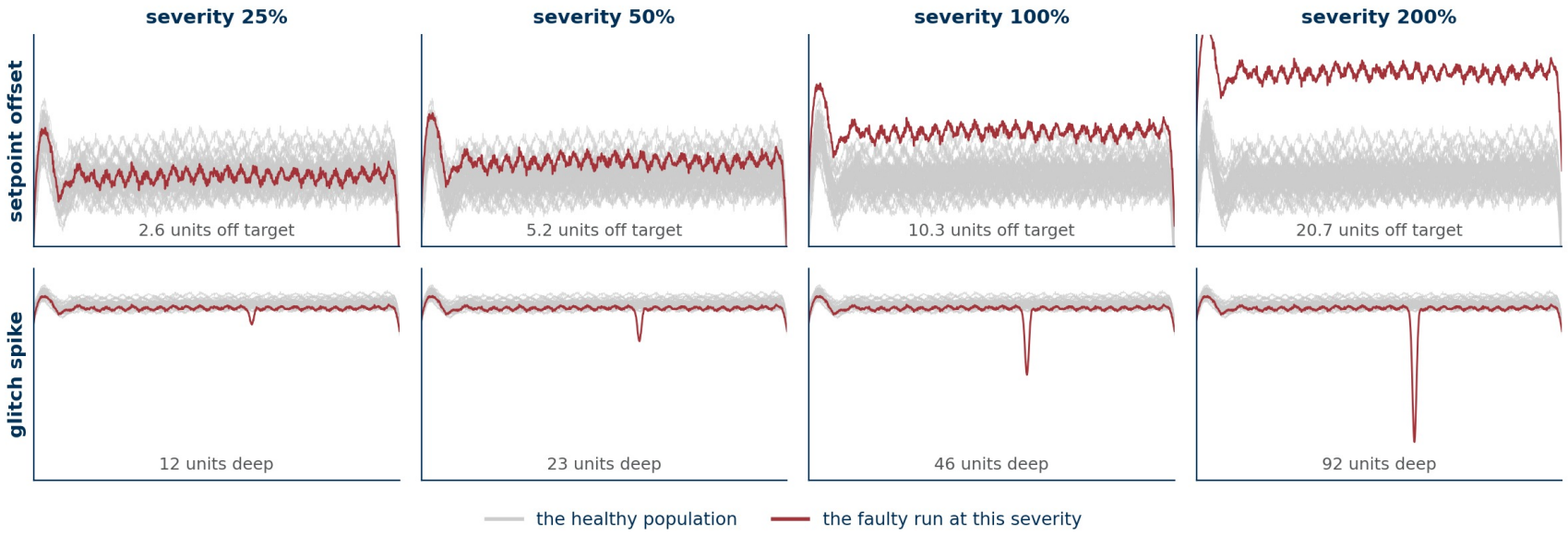
Good run, passed	**correct**	The 99% of runs nobody should hear about.
Good run, flagged	**false alarm**	Cheap once. Expensive in bulk, for reasons on the slide after next.
Faulty run, caught	**correct**	The entire point of the system.
Faulty run, passed	**escape**	The wafers ship. This is the outcome the fab is paying to avoid.

A detector is a choice about how to trade the second box against the fourth. Moving the limit trades one for the other and cannot reduce both.

Georgia Tech

What severity means

Every fault type has a nominal magnitude. Severity is that magnitude multiplied by a number, and 100% is the value used everywhere so far.



One knob per fault type

The defect keeps its shape and its location and changes only its size. At 25% the setpoint run sits inside the healthy band; at 200% it is nowhere near it.

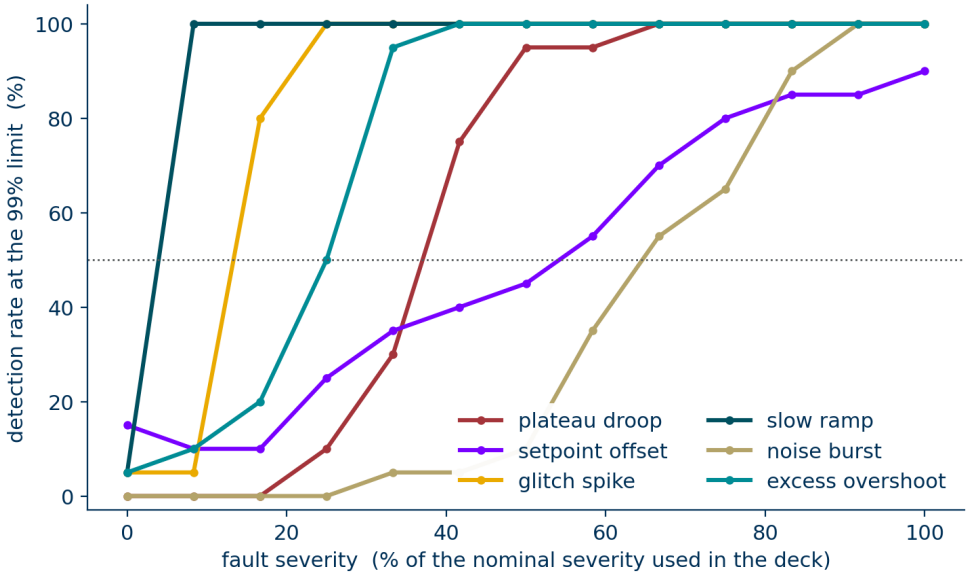
The nominal magnitudes

Slow ramp 2x longer · setpoint offset 4.5% of span · plateau droop 3% of span · overshoot 3.2x · glitch spike 20% of span · noise burst 6x noise.

Georgia Tech

How large does a fault have to be

Sweep each fault type from zero severity up to its nominal value and re-measure detection at the same limits.



Reading the ordering

Edge faults are easy. A ramp 10% slower is caught every time. The derivative is large there, so a small change is loud.

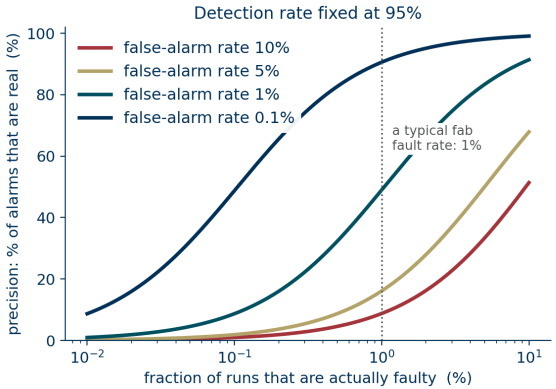
Plateau faults are hard. Setpoint offset and noise burst need most of their nominal severity before detection is reliable.

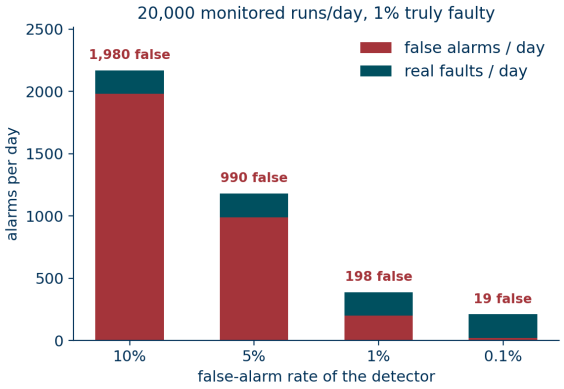
Report the floor, not the headline. A single detection rate hides all of this.

Georgia Tech

The arithmetic that decides whether the system stays on

Detection rate and false-alarm rate are properties of the detector. What an engineer sees also depends on the fab.





The three rates TP faulty, caught · FN faulty, passed · FP healthy, flagged · TN healthy, passed

detection rate	= faults caught / all faulty runs	`= TP / (TP + FN)`
false-alarm rate	= healthy flagged / all healthy runs	`= FP / (FP + TN)`
precision	= real alarms / all alarms raised	`= TP / (TP + FP)`

The first two are properties of the detector. Precision is not: it also depends on how often faults happen.

A worked case

20,000 monitored runs per day.

1% of them genuinely faulty, so 200 real faults.

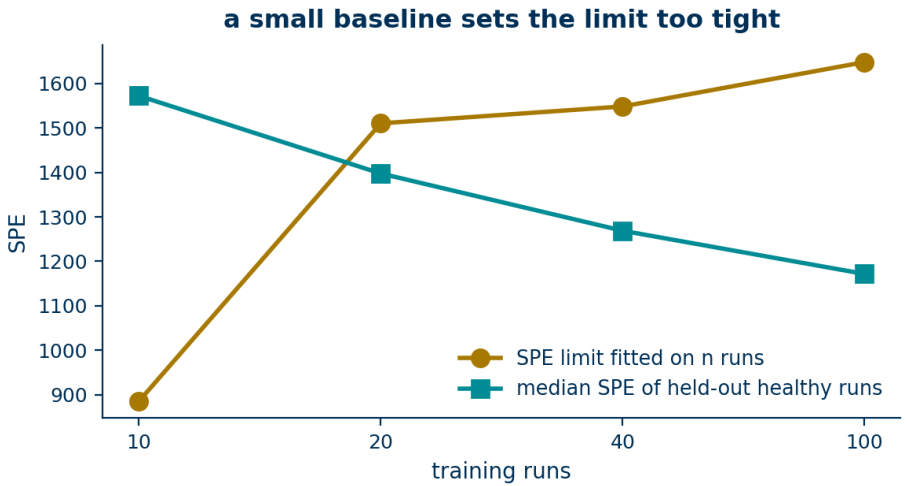
A detector at 95% detection and 5% false alarms.

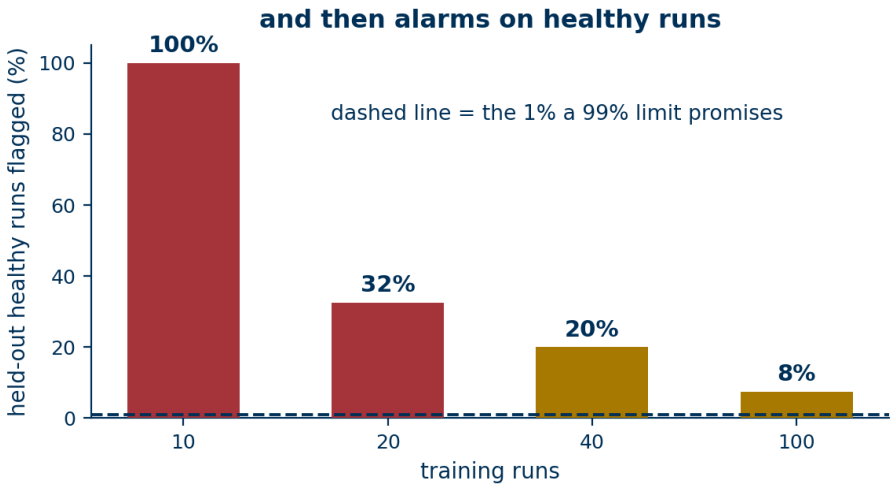
190 real alarms, and about 990 false ones. Five wrong for every one right.

Engineers mute an alarm that is wrong five times out of six, and then the system catches nothing at all.

How many runs the baseline needs

Fit the same model on 10, 20, 40 and 100 of the training runs, then score the 40 held-out healthy runs each time.





What goes wrong with few runs

With ten runs the model reconstructs its own training set almost perfectly, so the SPE limit comes out far too tight, and every one of the forty held-out runs breaks it.

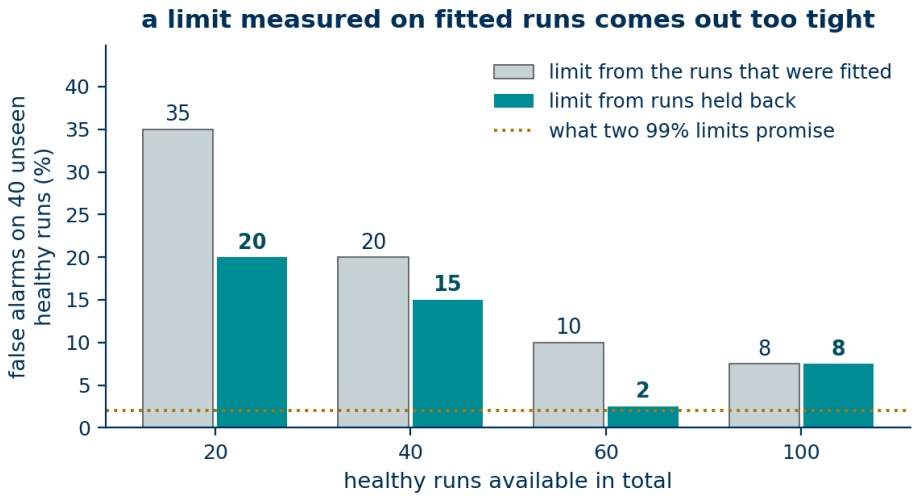
The general form

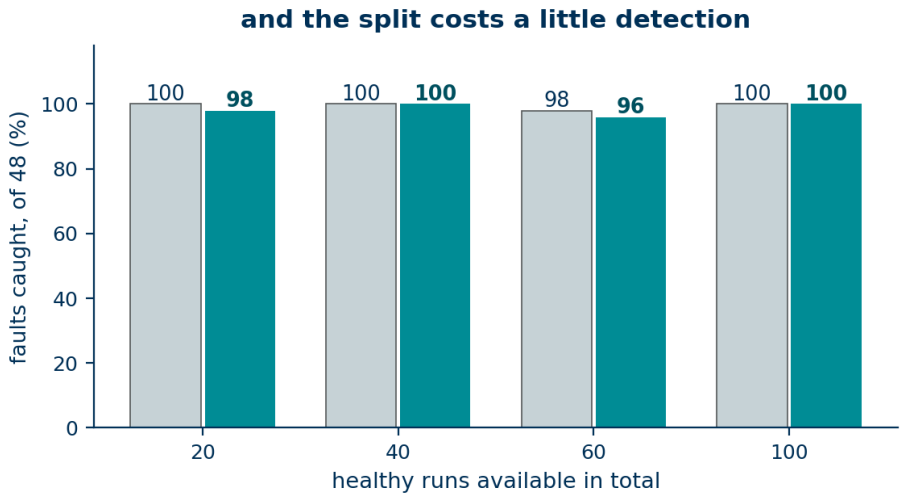
A model with k free directions fitted on n samples fits its own sample better than it fits new data. The gap closes as n grows, and at n = 100 it is down to 7.5%, which is the false-alarm rate on the held-out slide.

Georgia Tech

Where the limit is measured

The same healthy runs can fit the model or set the limit. Using them for both makes the limit too tight.





The split

Fit the mean and the eigentraces on part of the baseline. Score the runs you held back, and take the 99th percentile of those scores as the limit.

When it is worth doing

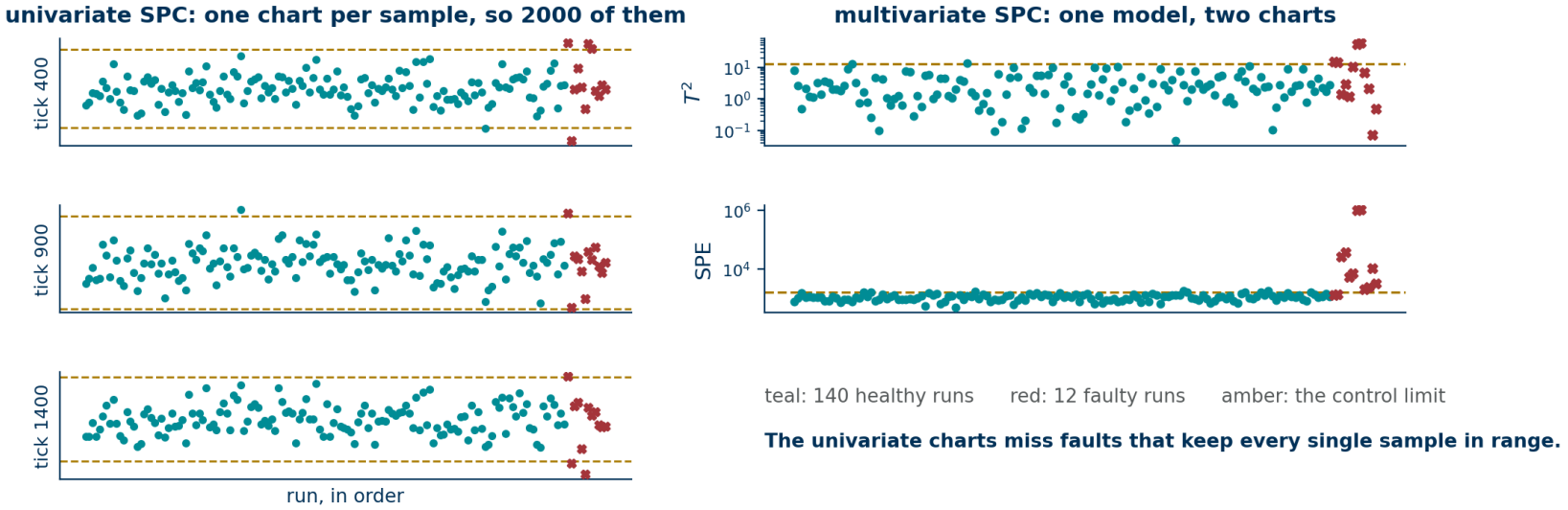
With twenty baseline runs the false-alarm rate falls from 35% to 20%. With a hundred there is nothing left to recover, and the smaller fit set costs a little detection.

Held-back runs are not a test set. They are part of the model, and they may not contain a fault.

Georgia Tech

What multivariate SPC means

Statistical process control charts a quantity against limits set from normal operation. The question is which quantity.



teal: 140 healthy runs red: 12 faulty runs amber: the control limit

The univariate charts miss faults that keep every single sample in range.

Why multivariate

A trace has 2000 samples, so univariate SPC means 2000 charts, and the samples are heavily correlated. Multivariate means one model of the whole run, and two charts.

Measured on this data

2000 charts at three sigma, alarm if any sample is out: 85% of faults caught, 28% of healthy runs flagged. T^2 and SPE: 100% caught, 7.5% flagged.

In those terms: the mean and the eigentraces are the model of normal operation, T^2 and SPE are the charted quantities, the 99th percentiles are the control limits, and the contribution plot is the diagnosis.

Georgia Tech

Where this method sits

Four tiers. Each one buys something the tier above cannot do, and gives something up.

Univariate statistical process control (SPC)	One control chart per extracted feature — plateau mean, rise time, overshoot, settling time. Shewhart charts the value itself, EWMA (exponentially weighted moving average) charts a running average, CUSUM (cumulative sum) charts accumulated departure from target.	Interpretable, almost no data needed. Blind to anything the summary statistic does not capture.
Multivariate SPC (MSPC) · today	A principal component analysis (PCA) baseline, scored with Hotelling's T^2 and the squared prediction error (SPE), plus contribution plots.	Handles the whole trace and many channels. Assumes one operating mode and a linear subspace.
Kernel and nonlinear PCA	The same construction carried out in a feature space, or a mixture of local linear models.	Handles curved and multi-mode baselines. Harder to interpret, more to tune.
Learned encoders	Autoencoders, sequence models, self-supervised pretraining on unlabelled runs.	Scales to many correlated channels and nonlinear couplings. Needs volume, compute and monitoring of its own.

Georgia Tech

Everything so far was already an autoencoder

A linear autoencoder trained to minimise squared reconstruction error recovers the principal subspace.

one run, 2000 numbers → **encode** → **the code, 3 numbers** → **decode** → **rebuilt run, 2000 numbers**

$$t = P^T (x - \bar{x}) \qquad \hat{x} = \bar{x} + P\,t \qquad SPE = \| x - \hat{x} \|^2$$

The mapping is exact

The encoder is the projection onto the eigentraces.

The code is the vector of coefficients.

The reconstruction loss is SPE, the quantity we are already monitoring.

What this buys you

Deep anomaly detection on traces is this construction with a nonlinear encoder and decoder. The scoring logic, the control limit and the contribution plot all carry over unchanged.

Georgia Tech

When a learned encoder is worth it

The useful question is which problem you have, rather than which method is better.

Where it wins

Many correlated channels. Dozens of sensors with nonlinear couplings, where hand-built features stop scaling.

Large unlabelled archives. Millions of healthy runs is exactly the setting self-supervised methods were built for.

Multiple operating modes. Several recipes on one tool, where a single linear subspace does not fit.

What it costs

A second system to monitor. The encoder itself drifts, and needs its own re-training policy.

Harder diagnosis. Contribution plots exist for nonlinear models but are less direct.

Weak baselines flatter it. Compare against aligned MSPC before claiming an improvement.

Georgia Tech

What breaks a deployed FDC system

None of these are modelling problems, and all of them will land on whoever builds one.

Post-maintenance drift	After preventive maintenance the tool legitimately changes and every trace shifts. Someone has to decide when to re-baseline: too eager and the model learns the fault, too slow and it alarms for a week.
Chamber mismatch	Nominally identical chambers differ. One baseline per chamber multiplies the number of models you maintain.
Recipe changes	A new recipe invalidates the baseline immediately, and recipe edits are routine.
Alarm fatigue	See the base-rate slide. A system that is muted is a system that detects nothing.

Summary

Five results from these two lectures.

1. **Two ways to be abnormal, two statistics.**
   T^2 catches a familiar ingredient in an unusual amount. SPE catches a shape the healthy population never produced. On the six fault types, setpoint offset is caught by T^2 alone and the glitch spike by SPE alone.

2. **The limit comes from the training scores.**
   The 99th percentile of the fitted runs, which retires the hand-picked threshold from Lecture 1 and re-derives itself whenever the baseline is re-fitted.

3. **Fit on good runs, and get the sample size right.**
   With ten training runs the SPE limit is too tight and every held-out healthy run alarms. With a hundred it is down to 7.5%, and it keeps falling as the baseline grows.

4. **A perfect score needs context.**
   48 of 48 caught and 3 of 40 falsely flagged is real, and it is measured on one injected fault per run at nominal severity. The severity floors and the base-rate arithmetic are the two numbers that decide whether it survives contact with a fab.

5. **The AI version is the same construction.**
   Encode, decode, and monitor what the decoder could not rebuild. A nonlinear encoder changes the first two steps and leaves the third alone.

Lab, and where the data came from

The notebook regenerates every figure in both decks from fixed seeds.

This week

Generate the healthy population and reproduce both Lecture 1 failures.

Implement landmark alignment and re-measure.

Fit the MSPC baseline, sweep k and the limit percentile, and report the severity floor for each fault type.

Honest caveats

The traces are simulated. Realistic magnitudes, one injected fault per run, no drift and no mode changes.

AUC of 0.999 is a property of that benchmark. It is why the severity floors and the base-rate slide exist.

Public datasets worth trying: SECOM, the Tennessee Eastman process, and the SEMI-PCB and WM-811K wafer-map sets for the classification half of FDC.

Georgia Tech